SGMS Scenario Management Microservice API Contract

Scenario Analysis and Simulation Service

Version 1.0.0

January 5, 2026

1. Introduction

The SGMS Scenario Management Microservice enables users to create, manage, and analyze electrical grid scenarios for simulation and contingency analysis. This service creates immutable snapshots of grid configurations and allows users to define events for scenario-based analysis.

1.1 Key Concepts

- **Scenario:** An immutable snapshot of a grid at a specific point in time, used for analysis
- **Grid Snapshot:** A read-only copy of grid topology (nodes and edges) captured when scenario is created
- **Event:** A defined change or occurrence in the grid (e.g., line outage, load change)
- **Scenario Status:** Lifecycle states (DRAFT, LOCKED, ARCHIVED) controlling scenario modifications

1.2 Base URL

All endpoints are relative to the base URL of the scenario management service.

1.3 Authentication

The API uses JWT (JSON Web Tokens) for authentication. All endpoints (except the welcome endpoint) require a cookie header with value:

```
cookie: <access_token>
```

2. API Endpoints

2.1 Welcome Endpoint

- **Endpoint:** `GET /`
- **Description:** Welcome message endpoint.
- **Response:** 200 OK with welcome message

2.2 Scenario Management

2.2.1 Create Scenario

- **Endpoint:** `POST /api/v1/scenarios/{grid_id}`
- **Description:** Create a new scenario from a base grid. Creates an immutable snapshot of the grid.
- **Key Features:**
  - Copies full grid data into an immutable snapshot
  - Does NOT create events (events must be added separately)
  - Scenario starts in DRAFT status
- **Parameters:**
  - `grid_id` (UUID): Base grid identifier
- **Request Body:**

```
{
    "name": "string"
}
```

- **Responses:**
  - **201 Created:** Scenario created successfully
  - **422 Validation Error:** Invalid input data

2.2.2 View Scenario

- **Endpoint:** `GET /api/v1/scenarios/{scenario_id}`
- **Description:** Fetch scenario with read-only grid snapshot and events. Frontend can render scenario topology for analysis.
- **Parameters:**
  - `scenario_id` (UUID): Scenario identifier
- **Response:** 200 OK with scenario details including grid snapshot and events

2.2.3 List Scenarios by Grid

- **Endpoint:** `GET /api/v1/scenarios/{grid_id}/list`
- **Description:** Retrieve all scenarios created from a specific grid
- **Parameters:**
  - `grid_id` (UUID): Base grid identifier
- **Response:** 200 OK with list of scenarios

2.2.4 Update Scenario Status

- **Endpoint:** `PATCH /api/v1/scenarios/{scenario_id}/status`
- **Description:** Change scenario lifecycle status
- **Parameters:**
  - `scenario_id` (UUID): Scenario identifier
- **Request Body:**

```
{
    "new_status": "DRAFT|LOCKED|ARCHIVED"
}
```

- **Response:** 200 OK with updated scenario details

2.2.5 Delete Scenario

- **Endpoint:** `DELETE /api/v1/scenarios/{scenario_id}`
- **Description:** Remove a scenario
- **Parameters:**
  - `scenario_id` (UUID): Scenario identifier
- **Response:** 200 OK

2.3 Event Management

2.3.1 Add Scenario Event

- **Endpoint:** `POST /api/v1/scenarios/{scenario_id}/events`
- **Description:** Add an event to a scenario for simulation purposes
- **Parameters:**
  - `scenario_id` (UUID): Scenario identifier
- **Request Body:**

```
{
    "event_type": "LOAD_CHANGE|LINE_OUTAGE|GENERATION_CHANGE|LOAD_SHEDDING|
    ISLANDING",
    "target_type": "NODE|EDGE|ZONE",
    "target_id": "UUID",
    "parameters": "object (optional)",
    "start_time": "datetime",
    "duration": "duration"
}
```

- **Responses:**
  - **201 Created:** Event added successfully
  - **422 Validation Error:** Invalid input data

3. Data Models

3.1 Scenario Models

3.1.1 ScenarioCreate

```
{
    "name": "string"
}
```

3.1.2 ScenarioOut

```
{
    "id": "UUID",
    "name": "string",
    "workspace_id": "UUID",
    "base_grid_id": "UUID",
    "status": "DRAFT|LOCKED|ARCHIVED",
    "created_by": "UUID",
    "created_at": "datetime"
}
```

3.1.3 ScenarioDetailOut

```
{
    "id": "UUID",
    "name": "string",
    "workspace_id": "UUID",
    "base_grid_id": "UUID",
    "status": "DRAFT|LOCKED|ARCHIVED",
    "created_by": "UUID",
    "created_at": "datetime",
    "grid_snapshot": {
        "id": "UUID",
        "scenario_id": "UUID",
        "nodes": "array",
        "edges": "array",
        "grid_metadata": "object (optional)",
        "created_at": "datetime"
    },
    "events": "array of ScenarioEventOut"
}
```

3.1.4 ScenarioListItem

```
{
    "id": "UUID",
    "name": "string",
    "status": "DRAFT|LOCKED|ARCHIVED",
    "created_at": "datetime"
}
```

3.2 Event Models

3.2.1 ScenarioEventCreate

```
{
    "event_type": "LOAD_CHANGE|LINE_OUTAGE|GENERATION_CHANGE|LOAD_SHEDDING|
    ISLANDING",
    "target_type": "NODE|EDGE|ZONE",
    "target_id": "UUID",
    "parameters": "object (optional)",
    "start_time": "datetime",
    "duration": "duration"
}
```

3.2.2 ScenarioEventOut

```
{
    "id": "UUID",
    "scenario_id": "UUID",
    "event_type": "LOAD_CHANGE|LINE_OUTAGE|GENERATION_CHANGE|LOAD_SHEDDING|
    ISLANDING",
    "target_type": "NODE|EDGE|ZONE",
    "target_id": "UUID",
    "parameters": "object (optional)",
    "start_time": "datetime",
    "duration": "duration",
    "created_at": "datetime"
}
```

3.3 Grid Snapshot Model

3.3.1 ScenarioGridSnapshotOut

```
{
    "id": "UUID",
    "scenario_id": "UUID",
    "nodes": "array of grid nodes",
    "edges": "array of grid edges",
    "grid_metadata": "object (optional)",
    "created_at": "datetime"
}
```

4. Enumerations

4.1 ScenarioStatus

- **DRAFT:** Scenario can be modified (add/remove events)
- **LOCKED:** Scenario is read-only for analysis/simulation
- **ARCHIVED:** Scenario is preserved for historical reference

4.2 EventType

- **LOAD_CHANGE:** Change in load demand at a node
- **LINE_OUTAGE:** Transmission line failure/disconnection
- **GENERATION_CHANGE:** Change in power generation at a plant
- **LOAD_SHEDDING:** Intentional load reduction for grid stability
- **ISLANDING:** Grid separation into isolated sections

4.3 TargetType

- **NODE:** Event targets a specific grid node (plant, load, substation)
- **EDGE:** Event targets a specific transmission line
- **ZONE:** Event targets a geographical or logical zone

5. Scenario Lifecycle

5.1 Creation Phase

1. User selects a base grid for scenario creation
2. System creates immutable snapshot of grid topology
3. Scenario is created in DRAFT status
4. User can add/remove events

5.2 Analysis Phase

1. User locks scenario (status: LOCKED)
2. Scenario becomes read-only for simulation
3. Analysis engines can run simulations on the frozen grid state
4. Results are stored separately from the scenario

5.3 Archival Phase

1. After analysis, scenario can be archived
2. Archived scenarios are preserved for historical reference
3. No further modifications allowed

6. Event Parameters Guide

6.1 Parameter Validation Rules

All events are validated using the following rules:

- Physical targets (NODE, EDGE) must exist in the grid snapshot
- Logical targets (ZONE) must reference existing nodes in the snapshot
- All required parameters must be provided based on event type
- Parameter values must be of correct type and format
- Events can only be added to scenarios in DRAFT status

6.2 LOAD_CHANGE Event

- **Target Type:** NODE (must be a LOAD node)
- **Validation:** Target node must have type "load" in snapshot
- **Required Parameters:**

```
{
    "delta_mw": "number",
    "mode": "string",
    "reason": "string"
}
```

- **Parameter Details:**
  - `delta_mw`: Change in load demand (positive for increase, negative for decrease)
  - `mode`: "absolute" or "percentage" change
  - `reason`: Reason for load change (e.g., "weather", "time_of_day")
- **Example:**

```
{
    "delta_mw": 50.5,
    "mode": "absolute",
    "reason": "peak_hour_demand"
}
```

6.3 GENERATION_CHANGE Event

- **Target Type:** NODE (must be a PLANT node)
- **Validation:** Target node must have type "plant" in snapshot
- **Required Parameters:**

```
{
    "new_capacity_mw": "number",
    "plant_type": "string",
    "cause": "string"
}
```

- **Parameter Details:**
  - `new_capacity_mw`: New generation capacity in megawatts
  - `plant_type`: Type of power plant (e.g., "hydro", "solar", "thermal")
  - `cause`: Reason for generation change (e.g., "maintenance", "fuel_supply")
- **Example:**

```
{
    "new_capacity_mw": 200.0,
    "plant_type": "thermal",
    "cause": "unit_outage"
}
```

6.4 LINE_OUTAGE Event

- **Target Type:** EDGE
- **Validation:** Target edge must exist in snapshot
- **Required Parameters:**

```
{
    "new_status": "string",
    "reason": "string"
}
```

- **Parameter Details:**
  - `new_status`: New operational status ("disabled" or "failed")
  - `reason`: Cause of outage (e.g., "storm", "maintenance", "fault")
- **Example:**

```
{
    "new_status": "disabled",
    "reason": "storm_damage"
}
```

6.5 LOAD_SHEDDING Event

- **Target Type:** NODE (must be a LOAD node)
- **Validation:** Target node must have type "load" in snapshot
- **Required Parameters:**

```
{
    "shed_mw": "number",
    "priority_threshold": "integer",
    "policy": "string"
}
```

- **Parameter Details:**
  - `shed_mw`: Amount of load to shed in megawatts

  - `priority_threshold`: Maximum priority level to shed (1-10, where 1 is highest priority)
  - `policy`: Shedding policy (e.g., "rotating", "emergency", "planned")

- **Example:**

```
{
    "shed_mw": 100.0,
    "priority_threshold": 3,
    "policy": "emergency"
}
```

6.6 ISLANDING Event

- **Target Type:** ZONE
- **Validation:** All node IDs in parameters must exist in snapshot
- **Required Parameters:**

```
{
    "node_ids": "array of UUIDs",
    "mode": "string",
    "reason": "string"
}
```

- **Parameter Details:**
  - `node_ids`: Array of UUIDs identifying nodes to include in the island
  - `mode`: Islanding mode ("intentional", "unintentional", "test")
  - `reason`: Reason for islanding (e.g., "protection", "maintenance", "test")
- **Example:**

```
{
    "node_ids": [
        "550e8400-e29b-41d4-a716-446655440000",
        "550e8400-e29b-41d4-a716-446655440001",
        "550e8400-e29b-41d4-a716-446655440002"
    ],
    "mode": "intentional",
    "reason": "grid_stabilization"
}
```

6.7 Common Error Scenarios

- **Invalid Target Type:** Event type requires specific target type
  - Example: LOAD_CHANGE requires NODE target, not EDGE
- **Target Not Found:** Physical target not in grid snapshot
- **Node Type Mismatch:** Target node type doesn't match event requirements
  - Example: LOAD_CHANGE on plant node
- **Missing Parameters:** Required parameters not provided

- **Invalid UUID Format:** Node IDs in ISLANDING not valid UUIDs
- **Empty Node List:** ISLANDING with empty node_ids array
- **Non-existent Nodes:** ISLANDING references nodes not in snapshot

6.8 Validation Code Example

The following Python code demonstrates the validation logic:

```
def validate_event_parameters(
    event_type: EventType,
    target_type: TargetType,
    parameters: dict,
    snapshot: List[dict],  # snapshot.nodes or snapshot.edges
    target_id: UUID | str  # allow logical IDs
):
    """
    Validate parameters for a scenario event based on the event type.
    Supports logical targets (e.g., ZONE for ISLANDING).
    Raises HTTPException if invalid.
    """

    snapshot_dict = {UUID(item["id"]): item for item in snapshot}

    # Validate physical target existence ONLY when required
    if target_type in {TargetType.NODE, TargetType.EDGE}:
        try:
            target_uuid = UUID(str(target_id))
        except ValueError:
            raise HTTPException(status_code=400,
                detail="Invalid UUID for physical target")

        if target_uuid not in snapshot_dict:
            raise HTTPException(status_code=400,
                detail="Target does not exist in snapshot")

        target = snapshot_dict[target_uuid]
    else:
        # Logical target (ZONE, SYSTEM, etc.)
        target = None

    # Event-specific validation rules
    if event_type == EventType.LOAD_CHANGE:
        if target_type != TargetType.NODE or target.get("type") != "load":
            raise HTTPException(status_code=400,
                detail="LOAD_CHANGE can only apply to LOAD nodes")
        required = ["delta_mw", "mode", "reason"]

    elif event_type == EventType.GENERATION_CHANGE:
        if target_type != TargetType.NODE or target.get("type") != "plant":
            raise HTTPException(status_code=400,
                detail="GENERATION_CHANGE can only apply to PLANT nodes")
        required = ["new_capacity_mw", "plant_type", "cause"]

    elif event_type == EventType.LINE_OUTAGE:
        if target_type != TargetType.EDGE:
            raise HTTPException(status_code=400,
                detail="LINE_OUTAGE can only apply to edges")
```

```
        required = ["new_status", "reason"]

    elif event_type == EventType.LOAD_SHEDDING:
        if target_type != TargetType.NODE or target.get("type") != "load":
            raise HTTPException(status_code=400,
                detail="LOAD_SHEDDING can only apply to LOAD nodes")
        required = ["shed_mw", "priority_threshold", "policy"]

    elif event_type == EventType.ISLANDING:
        if target_type != TargetType.ZONE:
            raise HTTPException(status_code=400,
                detail="ISLANDING must use ZONE as target_type")

        required = ["node_ids", "mode", "reason"]

        node_ids = parameters.get("node_ids", [])
        if not node_ids:
            raise HTTPException(status_code=400,
                detail="node_ids cannot be empty")

        # Validate referenced nodes exist
        for nid in node_ids:
            try:
                if UUID(nid) not in snapshot_dict:
                    raise HTTPException(
                        status_code=400,
                        detail=f"Node {nid} does not exist in snapshot"
                    )
            except ValueError:
                raise HTTPException(
                    status_code=400,
                    detail=f"Invalid UUID in node_ids: {nid}"
                )

    else:
        raise HTTPException(status_code=400,
            detail=f"Unsupported event type {event_type}")

    # Required parameter enforcement
    for key in required:
        if key not in parameters:
            raise HTTPException(
                status_code=400,
                detail=f"Missing required parameter: {key}"
            )

    return True
```

7. Error Handling

7.1 HTTPValidationError

```
{
    "detail": [
        {
            "loc": ["string" or "integer"],
            "msg": "string",
```

```
            "type": "string"
        }
    ]
}
```

7.2 Common HTTP Status Codes

- **200 OK:** Request successful
- **201 Created:** Resource created successfully
- **400 Bad Request:** Invalid request parameters
- **401 Unauthorized:** Authentication required
- **403 Forbidden:** Insufficient permissions or scenario locked
- **404 Not Found:** Scenario or grid not found
- **422 Unprocessable Entity:** Validation error
- **500 Internal Server Error:** Server error

7.3 Business Logic Errors

- **Scenario Locked:** Cannot modify events in LOCKED or ARCHIVED scenarios
- **Invalid Grid:** Base grid must exist and be accessible
- **Invalid Target:** Event target must exist in grid snapshot
- **Time Conflict:** Events cannot overlap in time

8. Security Considerations

- All endpoints require JWT authentication (except welcome)
- Users can only access scenarios in workspaces they have access to
- Grid snapshots are read-only and immutable
- Event modifications are logged for audit purposes
- Scenario status controls modification permissions
- Archived scenarios preserve historical data integrity

9. Rate Limiting

API requests are subject to rate limiting to prevent abuse. The current limits are:

- Scenario creation: 5 requests per minute per user
- Event management: 10 requests per minute per user
- Scenario queries: 20 requests per minute per user
- Status updates: 3 requests per minute per user

10. Implementation Notes

- Grid snapshots capture complete grid state at creation time
- Events are stored with ISO 8601 datetime format
- Duration uses ISO 8601 duration format (e.g., PT1H30M)
- Scenario names must be unique within a workspace
- Events can only be added to DRAFT scenarios
- Grid snapshots include all node/edge properties for accurate simulation
- Parameters field is flexible for event-specific data
- Archived scenarios can be restored to LOCKED status if needed

11. Integration with Other Services

- **Grid Modelling Service:** Retrieves base grid for snapshot creation
- **Analysis Engine:** Uses LOCKED scenarios for simulation
- **Results Service:** Stores simulation results linked to scenarios
- **Workspace Service:** Manages workspace permissions and access